

TALLER DE ALGORITMOS DE ORDENAMIENTO

Informe de resultados

Integrantes: Andres Santiago Castro Barrero - 20251020073

Juan Alejandro Peña Barreto - 20251020023

Fecha: 02 / 09 / 2026

1. Metodología de las pruebas

Para cada tamaño de arreglo N (3.000 / 30.000 / 300.000 / 3.000.000 / 30.000.000) se generó un único conjunto base de números aleatorios, a partir del cual se derivaron las tres variantes exigidas por el taller, aplicadas por igual a los cinco algoritmos:

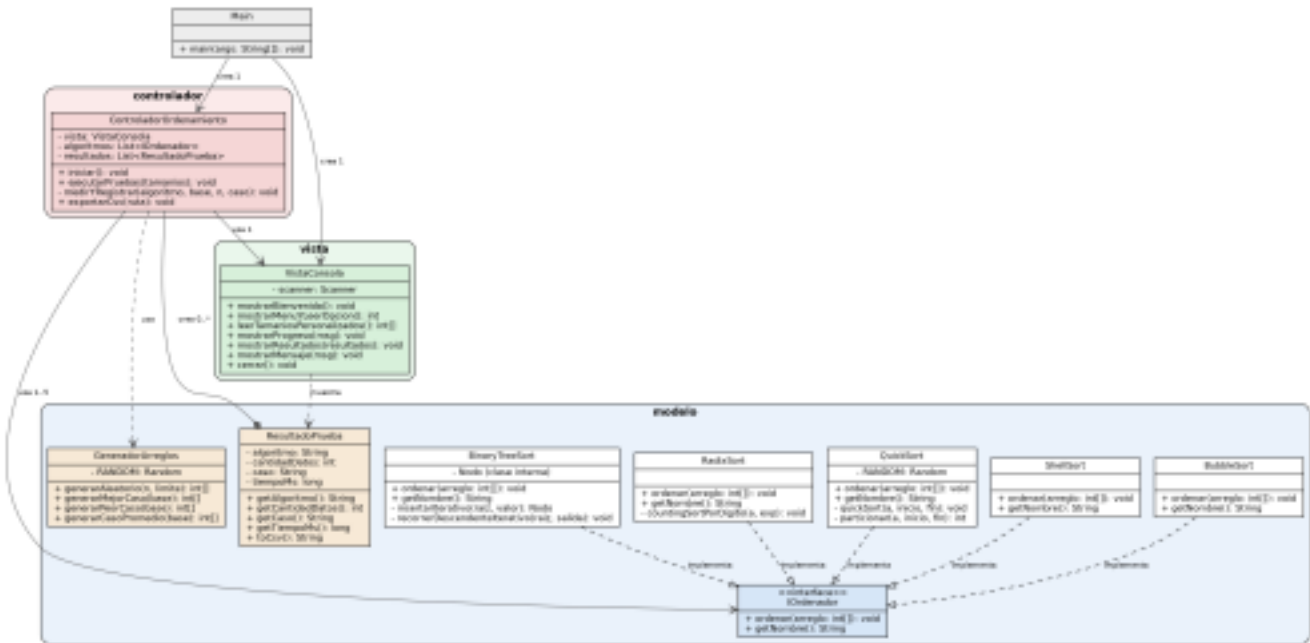
- **Mejor caso:** el arreglo ya está ordenado de forma descendente (el orden buscado).
- **Peor caso:** el arreglo está ordenado de forma ascendente (exactamente al revés).
- **Caso promedio:** el arreglo aleatorio original, sin ningún orden particular.

El tiempo se midió únicamente durante la ejecución del algoritmo de ordenamiento (no incluye la generación de los datos aleatorios ni la impresión de resultados), usando `System.nanoTime()` en Java y `time.perf_counter()` en Python, convertido a milisegundos.

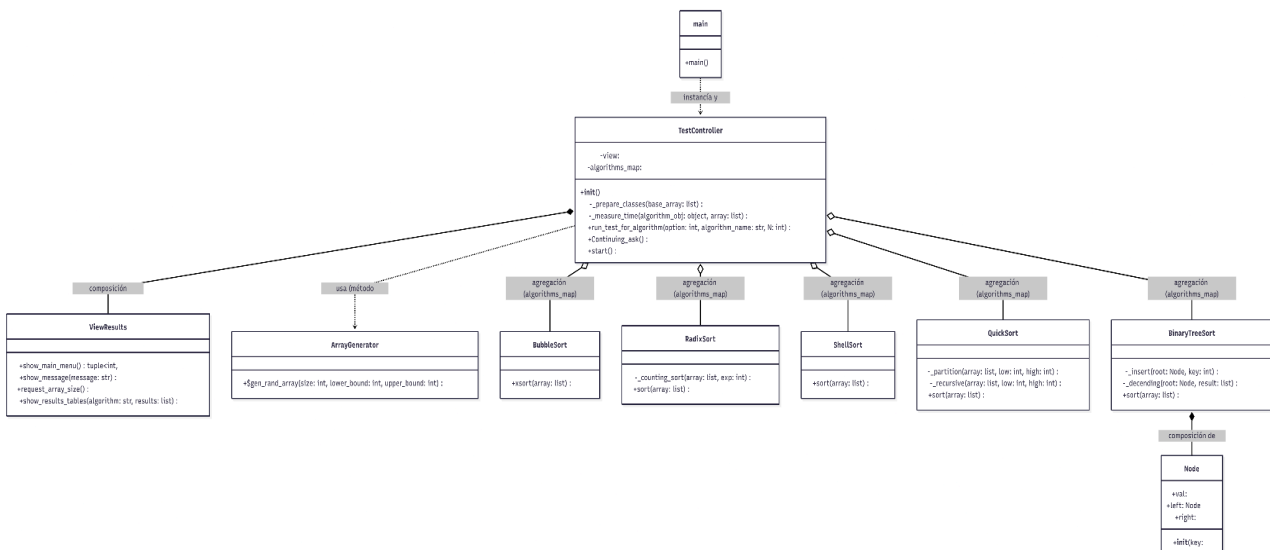
2. Diagrama de clases

2.1 proyecto Java, patrón MVC

El diagrama siguiente corresponde al proyecto Java. La interfaz `IOrdenador` define el contrato común (patrón Strategy) que implementan los cinco algoritmos; el Controlador coordina el Modelo y la Vista.



2.2 proyecto python, patron MVC



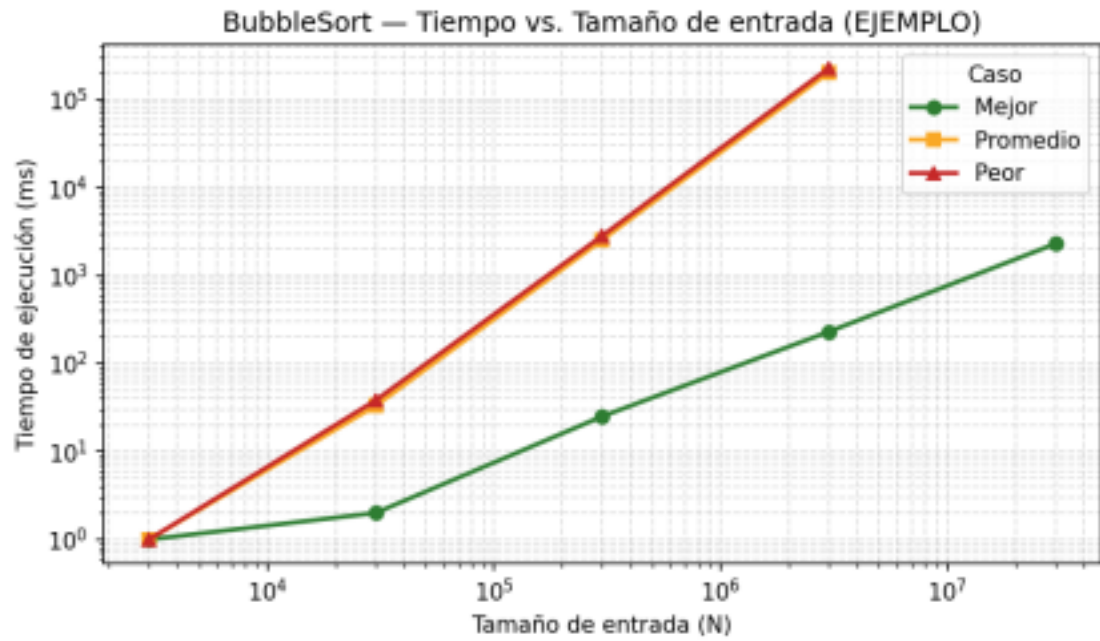
3. Resultados por algoritmo

Tabla y gráfica (esta última en escala log-log) de tiempo de ejecución vs. tamaño de entrada, para los tres casos, por cada algoritmo.

3.1 Implementación en JAVA

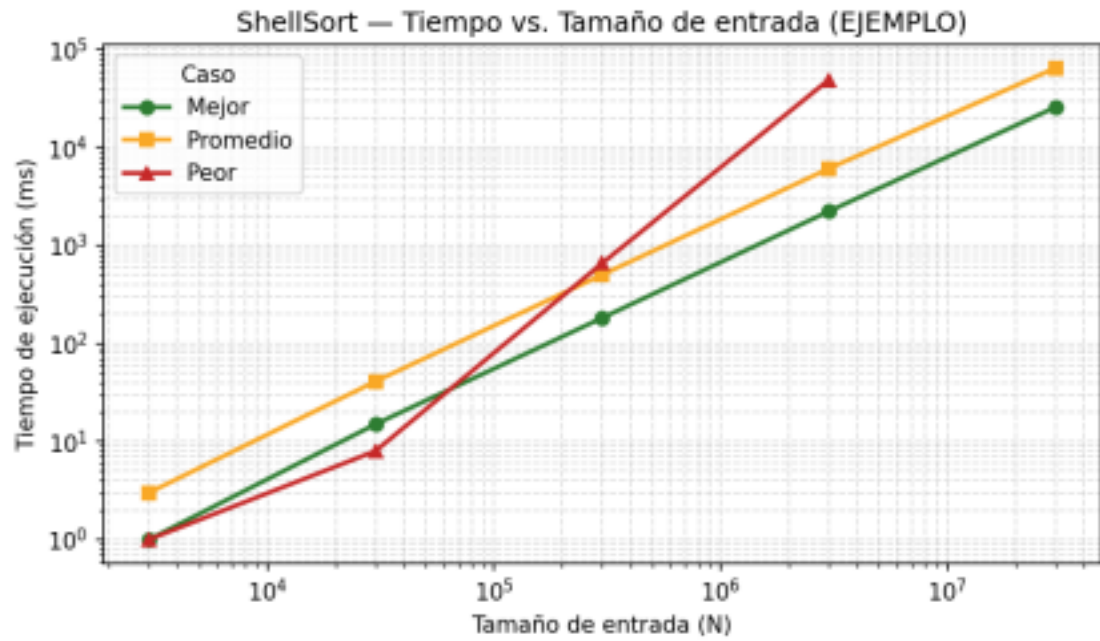
BubbleSort

3.000	1	Mejor
3.000	1	Peor
3.000	1	Promedio
30.000	2	Mejor
30.000	38	Peor
30.000	33	Promedio
300.000	25	Mejor
300.000	2.811	Peor
300.000	2.530	Promedio
3.000.000	229	Mejor
3.000.000	226.378	Peor
3.000.000	202.658	Promedio
30.000.000	2.286	Mejor



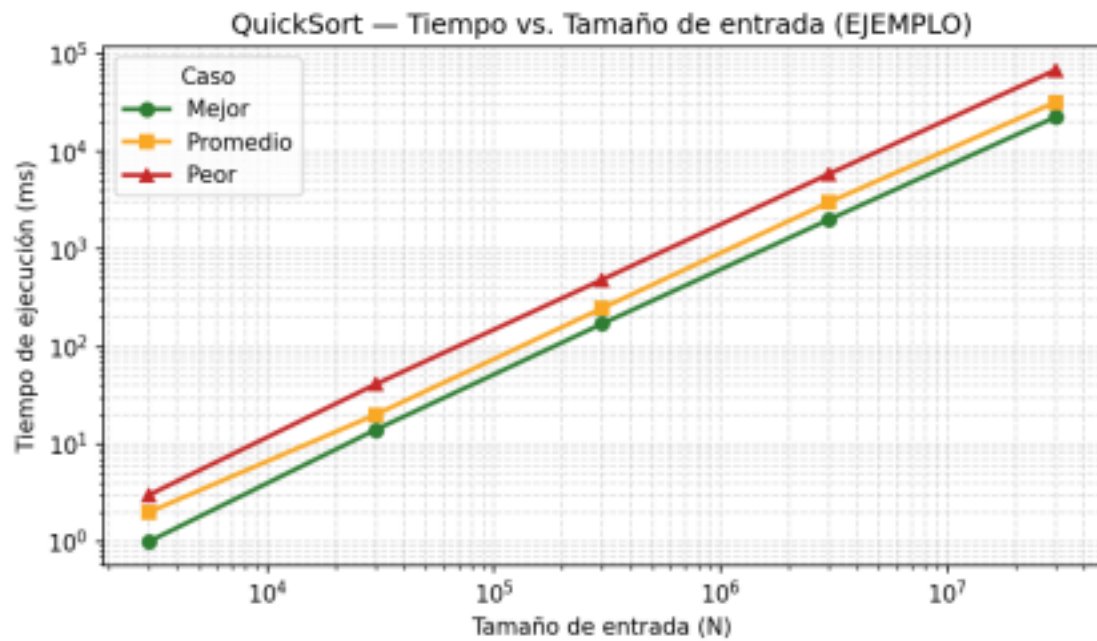
ShellSort

3.000	1	Mejor
3.000	1	Peor
3.000	3	Promedio
30.000	15	Mejor
30.000	8	Peor
30.000	41	Promedio
300.000	182	Mejor
300.000	657	Peor
300.000	501	Promedio
3.000.000	2.223	Mejor
3.000.000	48.893	Peor
3.000.000	6.075	Promedio
30.000.000	25.654	Mejor
30.000.000	64.333	Promedio



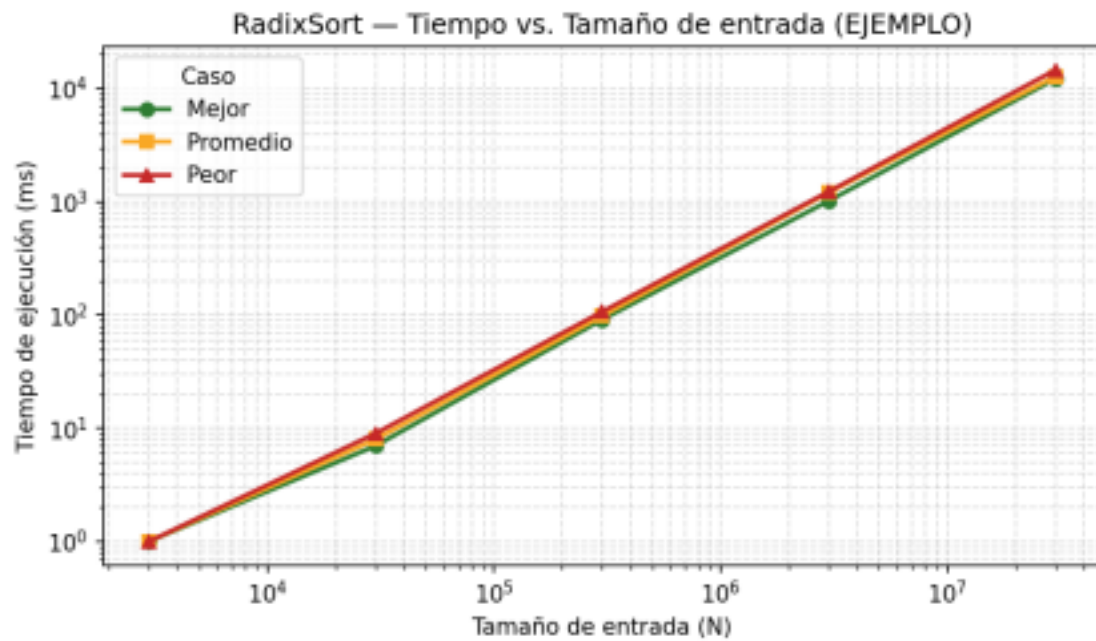
QuickSort

3.000	1	Mejor
3.000	3	Peor
3.000	2	Promedio
30.000	14	Mejor
30.000	41	Peor
30.000	20	Promedio
300.000	171	Mejor
300.000	485	Peor
300.000	247	Promedio
3.000.000	2.000	Mejor
3.000.000	5.878	Peor
3.000.000	3.010	Promedio
30.000.000	22.528	Mejor
30.000.000	68.436	Peor
30.000.000	32.009	Promedio



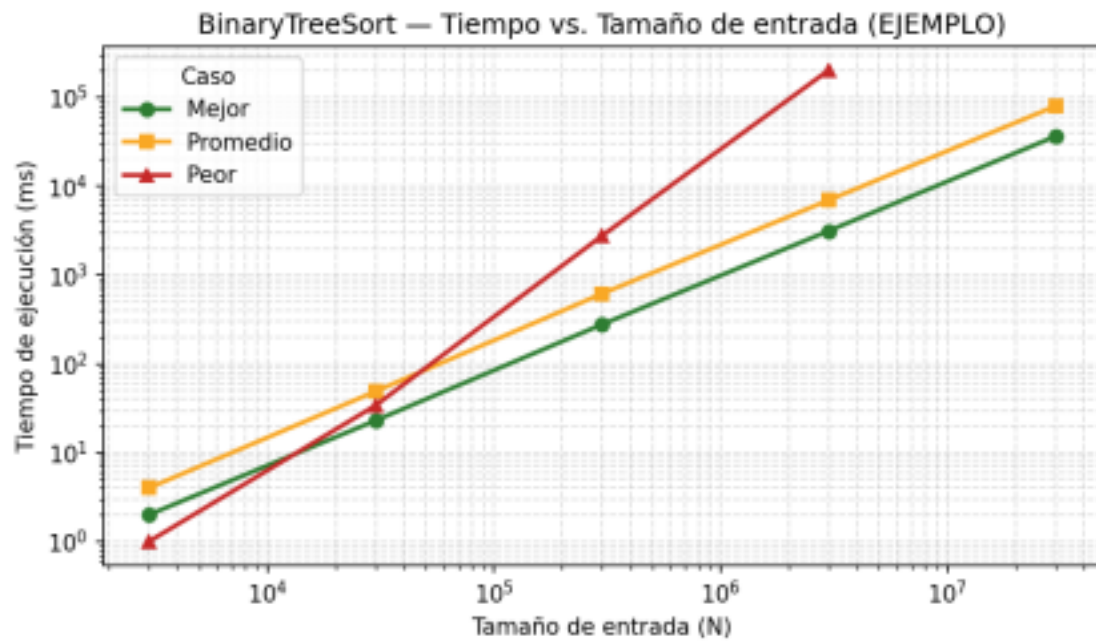
RadixSort

3.000	1	Mejor
3.000	1	Peor
3.000	1	Promedio
30.000	7	Mejor
30.000	9	Peor
30.000	8	Promedio
300.000	91	Mejor
300.000	107	Peor
300.000	98	Promedio
3.000.000	1.019	Mejor
3.000.000	1.231	Peor
3.000.000	1.205	Promedio
30.000.000	12.178	Mejor
30.000.000	14.557	Peor
30.000.000	12.766	Promedio



BinaryTreeSort

3.000	2	Mejor
3.000	1	Peor
3.000	4	Promedio
30.000	23	Mejor
30.000	34	Peor
30.000	49	Promedio
300.000	278	Mejor
300.000	2.769	Peor
300.000	617	Promedio
3.000.000	3.140	Mejor
3.000.000	202.725	Peor
3.000.000	6.969	Promedio
30.000.000	36.392	Mejor
30.000.000	79.598	Promedio



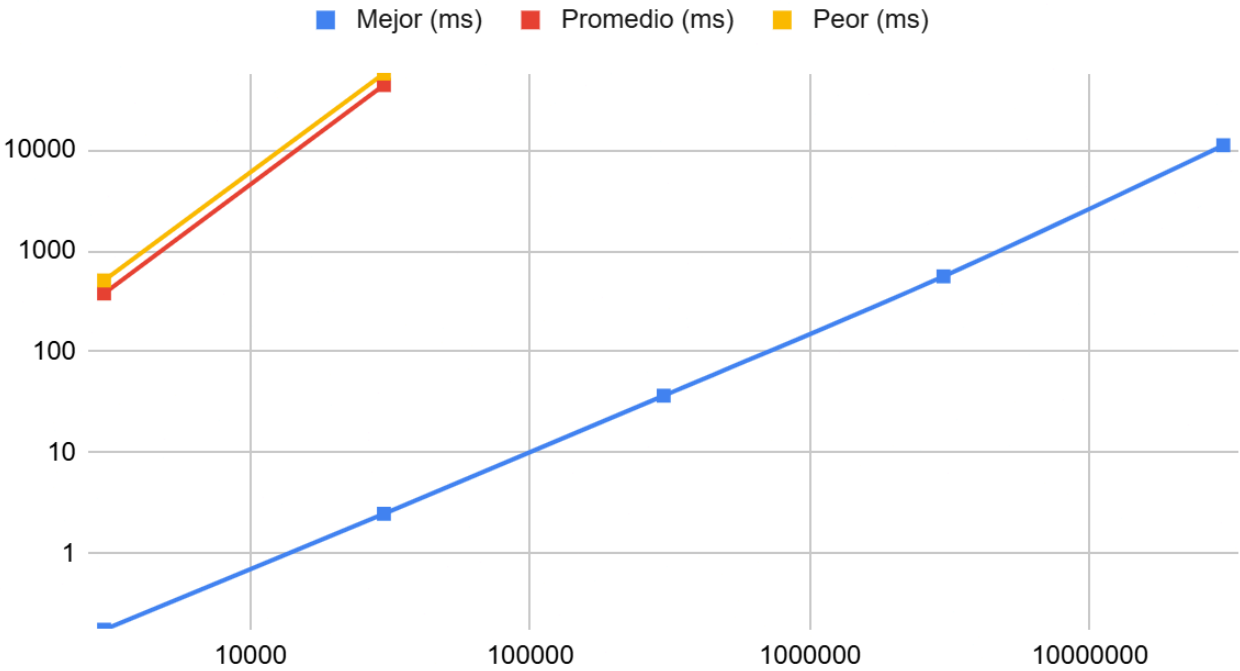
3.2 Implementación Python

Bubble Sort

Cantidad de Datos (N)	Caso	Tiempo (ms)
3000	Mejor	0.1759
3000	Promedio	378.9858
3000	Peor	509.722
30000	Mejor	2.4796
30000	Promedio	97895.0058
30000	Peor	166722.8682
300000	Mejor	36.7476
300000	Promedio	N/A (Took too long)
300000	Peor	N/A (Took too long)
3000000	Mejor	559.8881
3000000	Promedio	N/A (Took too long)
3000000	Peor	N/A (Took too long)

30000000	Mejor	11163.3109
30000000	Promedio	N/A (Took too long)
30000000	Peor	N/A (Took too long)

Cantidad de Datos (N), Mejor (ms), Promedio (ms) y Peor (ms)

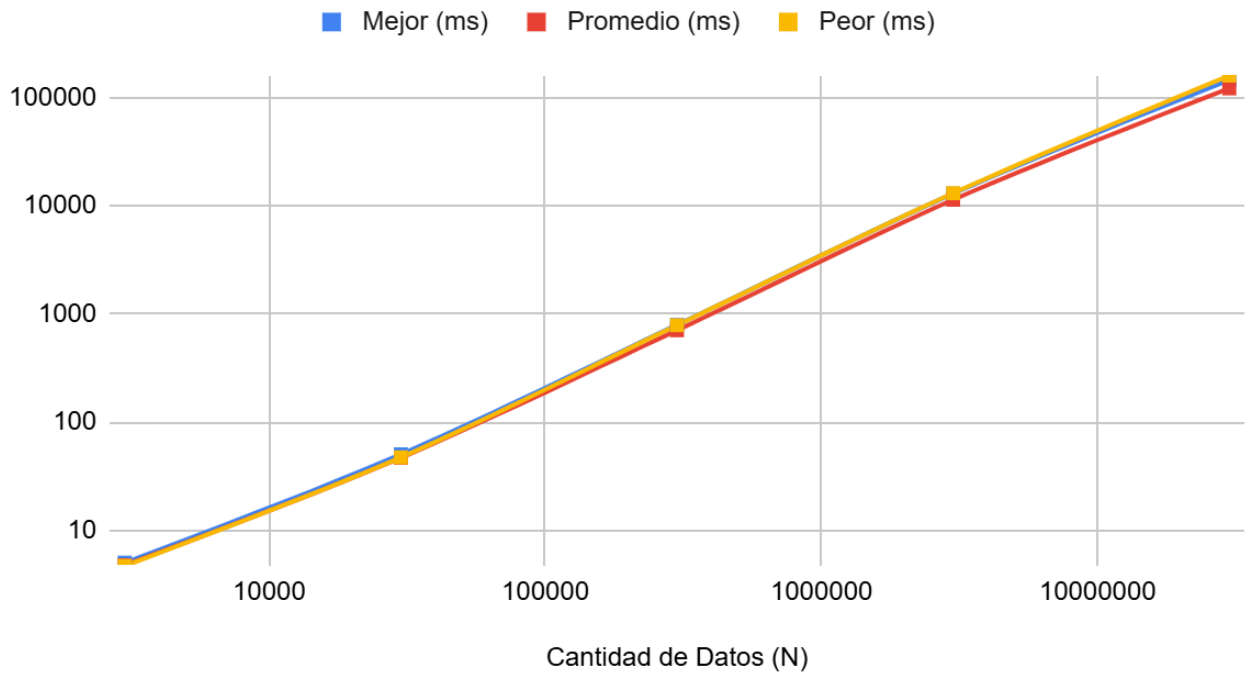


Radix Sort

Cantidad de Datos (N)	Caso	Tiempo (ms)
3000	Mejor	5.0543
3000	Promedio	4.7945
3000	Peor	4.7277
30000	Mejor	50.5984
30000	Promedio	47.0037
30000	Peor	47.4415
300000	Mejor	792.4544
300000	Promedio	706.8071
300000	Peor	786.9605
3000000	Mejor	12910.8567

3000000	Promedio	11330.0778
3000000	Peor	13022.0418
30000000	Mejor	144735.4436
30000000	Promedio	120899.691
30000000	Peor	158766.4746

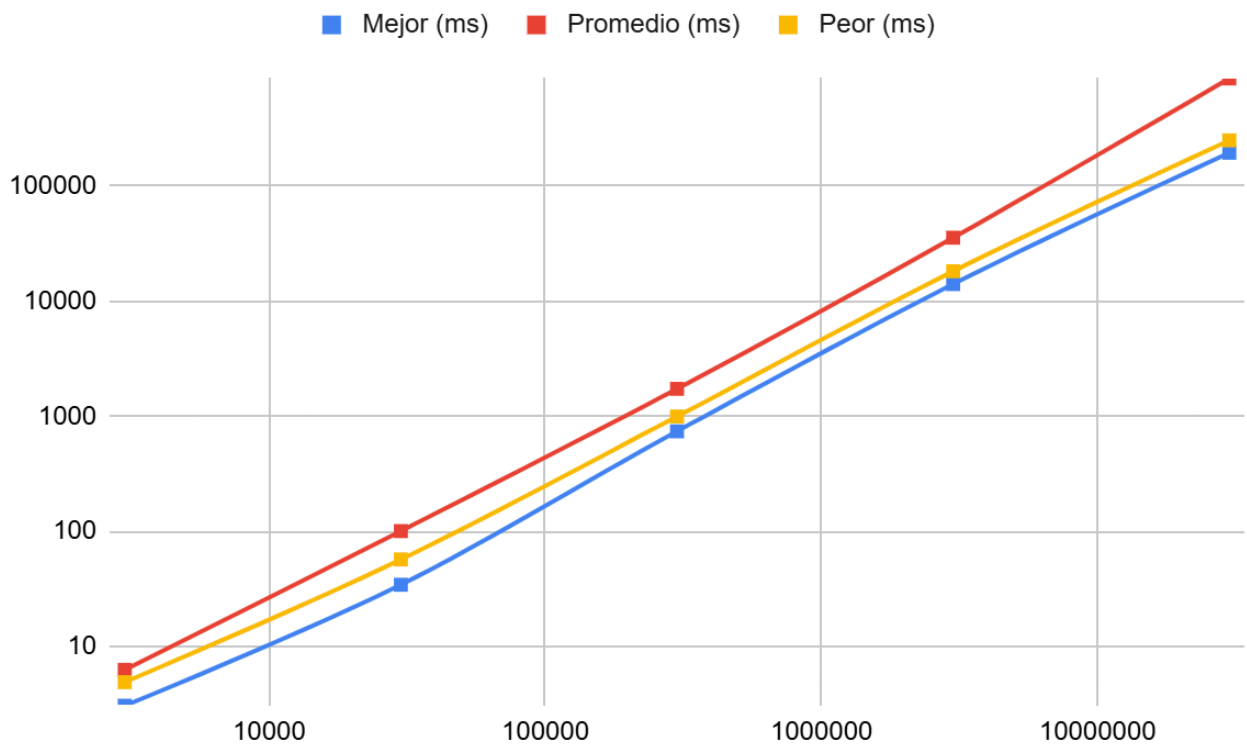
Mejor (ms), Promedio (ms) y Peor (ms)



Shell Sort

Cantidad de Datos (N)	Caso	Tiempo (ms)
3000	Mejor	3.0835
3000	Promedio	6.3068
3000	Peor	4.9343
30000	Mejor	34.5267
30000	Promedio	100.8302
30000	Peor	57.231
300000	Mejor	739.4629

300000	Promedio	1726.5594
300000	Peor	993.9604
3000000	Mejor	13983.3136
3000000	Promedio	35320.8342
3000000	Peor	18067.6855
30000000	Mejor	192568.7253
30000000	Promedio	849448.5311
30000000	Peor	246627.7593

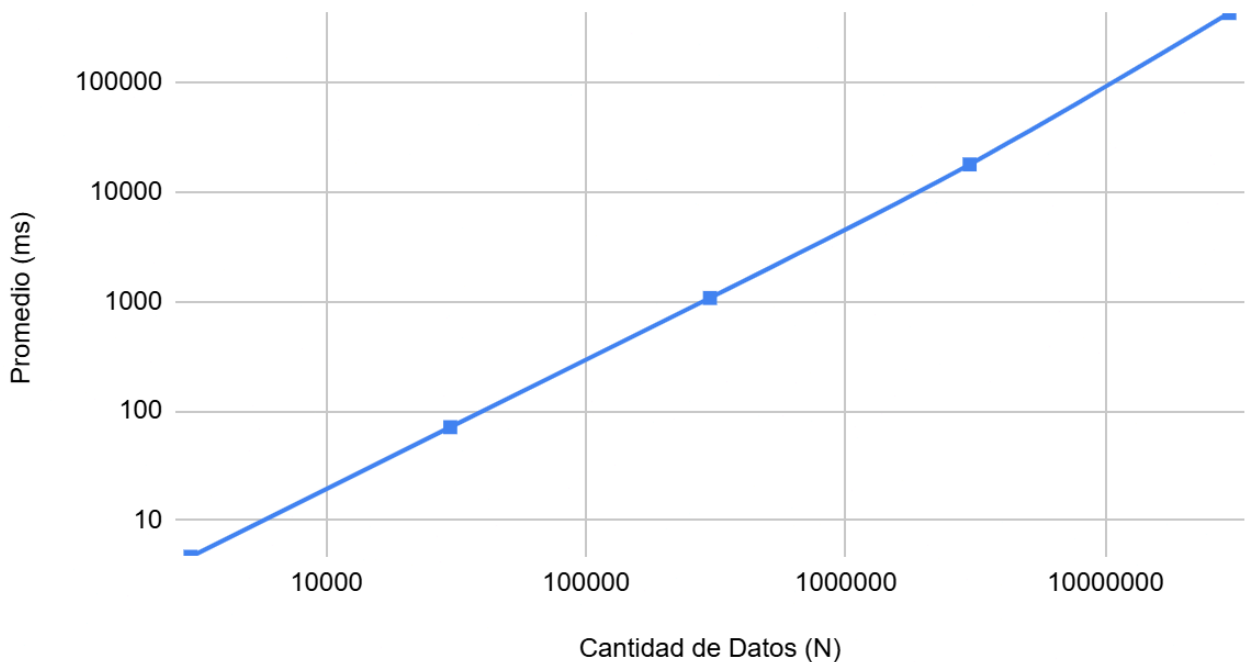


Binary Tree Sort

Cantidad de Datos (N)	Caso	Tiempo (ms)
3000	Mejor	N/A (Stack Overflow)
3000	Promedio	4.6624
3000	Peor	N/A (Stack Overflow)

30000	Mejor	N/A (Stack Overflow)
30000	Promedio	71.0682
30000	Peor	N/A (Stack Overflow)
300000	Mejor	N/A (Stack Overflow)
300000	Promedio	1074.2457
300000	Peor	N/A (Stack Overflow)
3000000	Mejor	N/A (Stack Overflow)
3000000	Promedio	17789.2055
3000000	Peor	N/A (Stack Overflow)
30000000	Mejor	N/A (Stack Overflow)
30000000	Promedio	428769.2278
30000000	Peor	N/A (Stack Overflow)

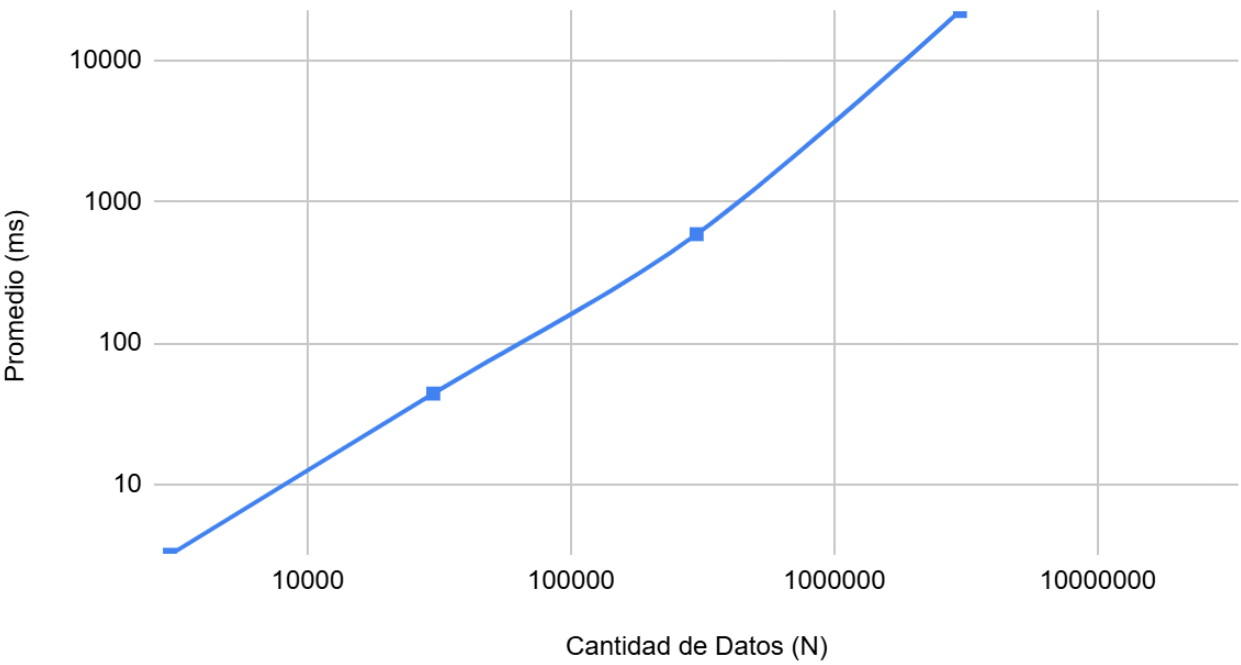
Promedio (ms) contra Cantidad de Datos (N)



Quick Sort

Cantidad de Datos (N)	Caso	Tiempo (ms)
3000	Mejor	N/A (Stack Overflow)
3000	Promedio	3.1611
3000	Peor	N/A (Stack Overflow)
30000	Mejor	N/A (Stack Overflow)
30000	Promedio	43.8326
30000	Peor	N/A (Stack Overflow)
300000	Mejor	N/A (Stack Overflow)
300000	Promedio	590.1262
300000	Peor	N/A (Stack Overflow)
3000000	Mejor	N/A (Stack Overflow)
3000000	Promedio	22474.6805
3000000	Peor	N/A (Stack Overflow)
30000000	Mejor	N/A (Took too long)
30000000	Promedio	N/A (Took too long)
30000000	Peor	N/A (Took too long)

Promedio (ms) contra Cantidad de Datos (N)



4. Preguntas

a. ¿Cuál algoritmo fue más fácil de implementar?

Para ambas implementaciones se evidencio particular facilidad al momento de implementar el algoritmos de ordenamiento de burbuja, al ser solo un recorrido por el arreglo que compara pares, se pudo simplemente anidar dos comparaciones, que fueran pasando por cada par de números del arreglo, para ambos proyectos, a este algoritmo se le añadió una “bandera de optimización” que consiste en un booleano que comprueba si hubo intercambio, para así obviar los casos ya ordenados

b. ¿Qué se entiende por mejor caso, peor caso y caso medio?

Mejor caso: la entrada específica para la cual el algoritmo requiere el menor tiempo posible (aquí, el arreglo que ya viene en el orden buscado). **Peor caso:** la entrada específica que maximiza el tiempo de ejecución (aquí, el arreglo en el orden exactamente opuesto). **Caso promedio:** el tiempo esperado considerando todas las entradas posibles con la misma probabilidad, aproximado en la práctica con un arreglo aleatorio. Estos tres casos describen cómo se comporta la complejidad de un algoritmo no solo en función del tamaño de la entrada (N), sino también de su distribución/orden inicial.

c. ¿Cuál algoritmo es más eficiente según las pruebas realizadas?

Acorde a los resultados tomados, el algoritmo más eficiente fue el Radix Sort, tanto en Python como en Java. Su principal fuerte es el hecho de que no es un algoritmo de comparación, lo que implica que su complejidad pasa de una del tipo logarítmica, a prácticamente reducirse de forma lineal, eso se refleja en el hecho de que fue uno de los pocos algoritmos que, para la implementación en Python, no presentó complicaciones del tipo falta de memoria o tiempo intratable, además de presentar estabilidad en los tiempos independientemente del caso, mostrando que no es tan afectado como los demás algoritmos evaluados, por las condiciones iniciales del arreglo presentado.

d. ¿Cuál algoritmo es menos eficiente según las pruebas realizadas?

El algoritmo menos eficiente fue el Bubble Sort, no solo por sus tiempos registrados, si no por su incapacidad de tratar con grupos grandes de datos, a partir de arreglos desordenados u ordenados al revés de tamaño $N = 300.000$, esto se explica por su necesidad rigurosa de comparar cada par de elementos adyacentes para efectuar el ordenamiento, sobrecargando el procesamiento del IDE y muchas veces simplemente, por mas que se espere, no presentando efectivamente los resultados de ordenamiento en un tiempo tratable.

e. ¿Qué recomendaciones darían a nuevas personas que hagan esta práctica?

1) probar primero con N pequeños (cientos o miles) antes de correr los tamaños grandes; 2) usar exactamente el mismo conjunto de datos base para los tres casos y para los cinco algoritmos, así la comparación es justa; 3) cerrar otros programas mientras se mide el tiempo, para no contaminar la medición; 4) para QuickSort, usar un pivote aleatorio (o mediana de tres) para evitar el peor caso teórico $O(n^2)$; 5) para BinaryTreeSort, implementar la inserción y el recorrido de forma iterativa (no recursiva) si N es grande, para evitar errores de desbordamiento de pila.

f. Hardware utilizado en el taller

Procesador: Intel(R) Core(TM) i7-8650U RAM: 16GB DDR4
Sistema operativo: Windows 11 Almacenamiento : SSD 512 GB NVMe

Dificultades presentadas durante la práctica